IMEN891K / IDSC501 / AXGS-501 · DATA ENGINEERING · FALL 2026

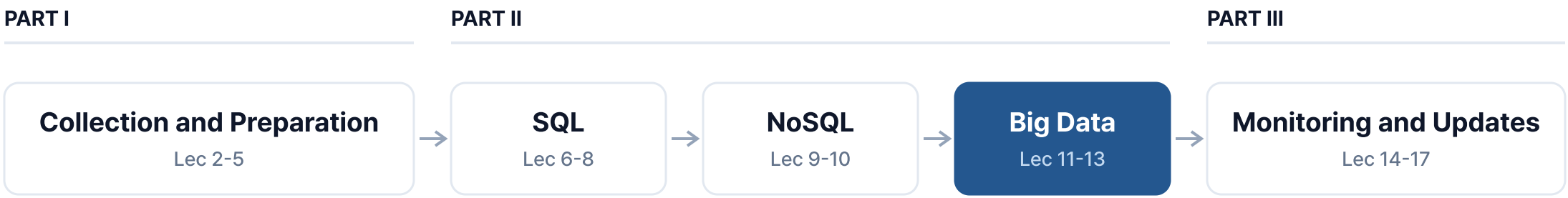
12.

Big Data II: Data Warehousing

Sangwoo Mo · POSTECH

Industrial & Management Engineering

Where We Are


PART I
PART II
PART III
Collection and Preparation
Lec 2-5
SQL
Lec 6-8
NoSQL
Lec 9-10
Big Data
Lec 11-13
Monitoring and Updates
Lec 14-17

Recap: Distributed Batch Processing

A distributed program

```
(sales.groupBy("product_id")
      .sum("amount")
      .show())
```

A distributed program still needs infrastructure, storage and a way to run again

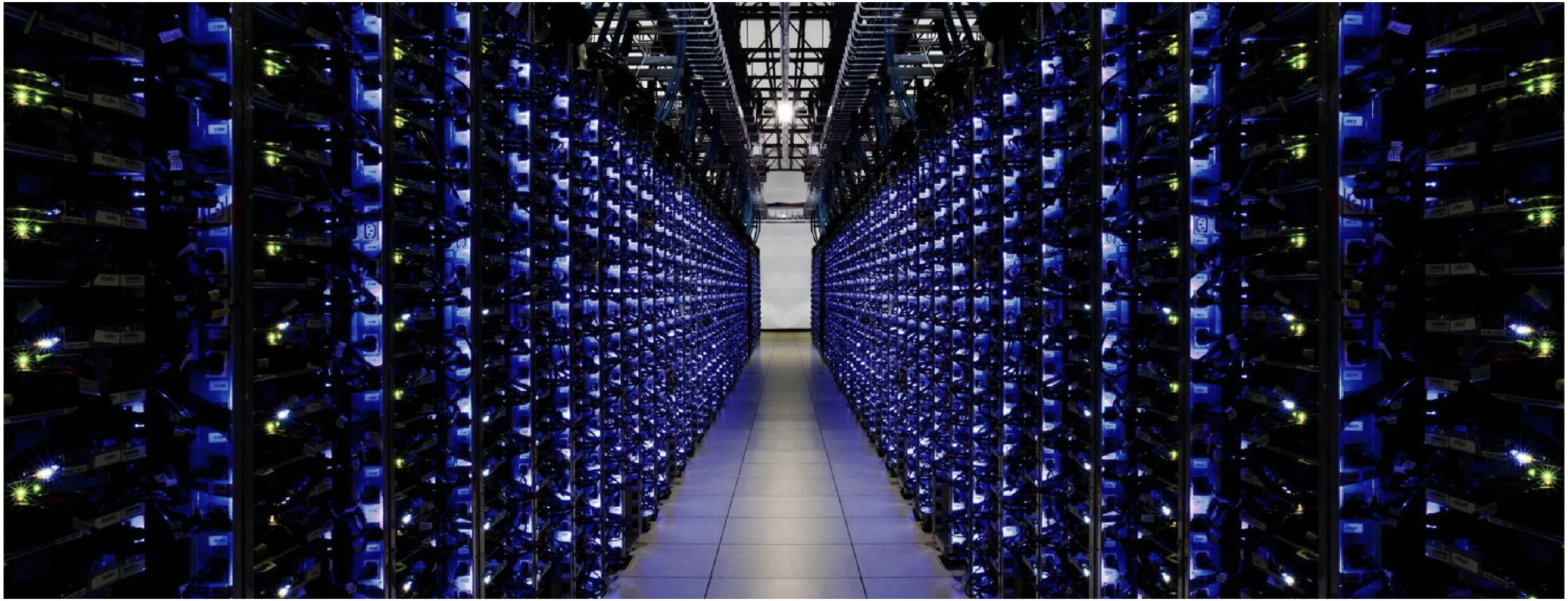
Outline: Big Data II

01	**Cloud and Serverless**	Who operates the infrastructure?
02	**Analytical Warehouses**	How are many rows read and summarized?
03	**Lakes and Lakehouses**	Where do files become tables?
04	**Data Orchestration**	Which task runs after which, and safely again?

01 · Cloud and Serverless

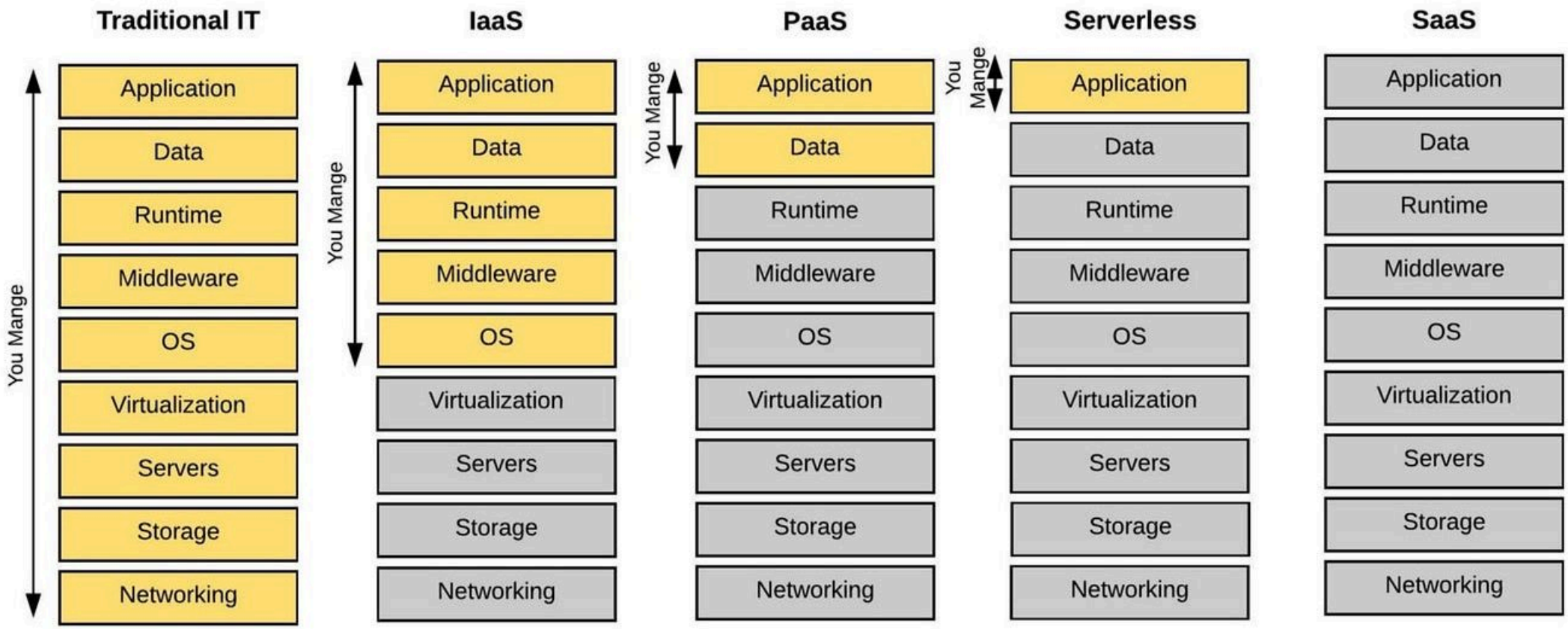
01	**Cloud and Serverless**	Who operates the infrastructure?
02	**Analytical Warehouses**	How are many rows read and summarized?
03	**Lakes and Lakehouses**	Where do files become tables?
04	**Data Orchestration**	Which task runs after which, and safely again?

Why Managed Infrastructure



The cloud still runs on physical servers that somebody must operate

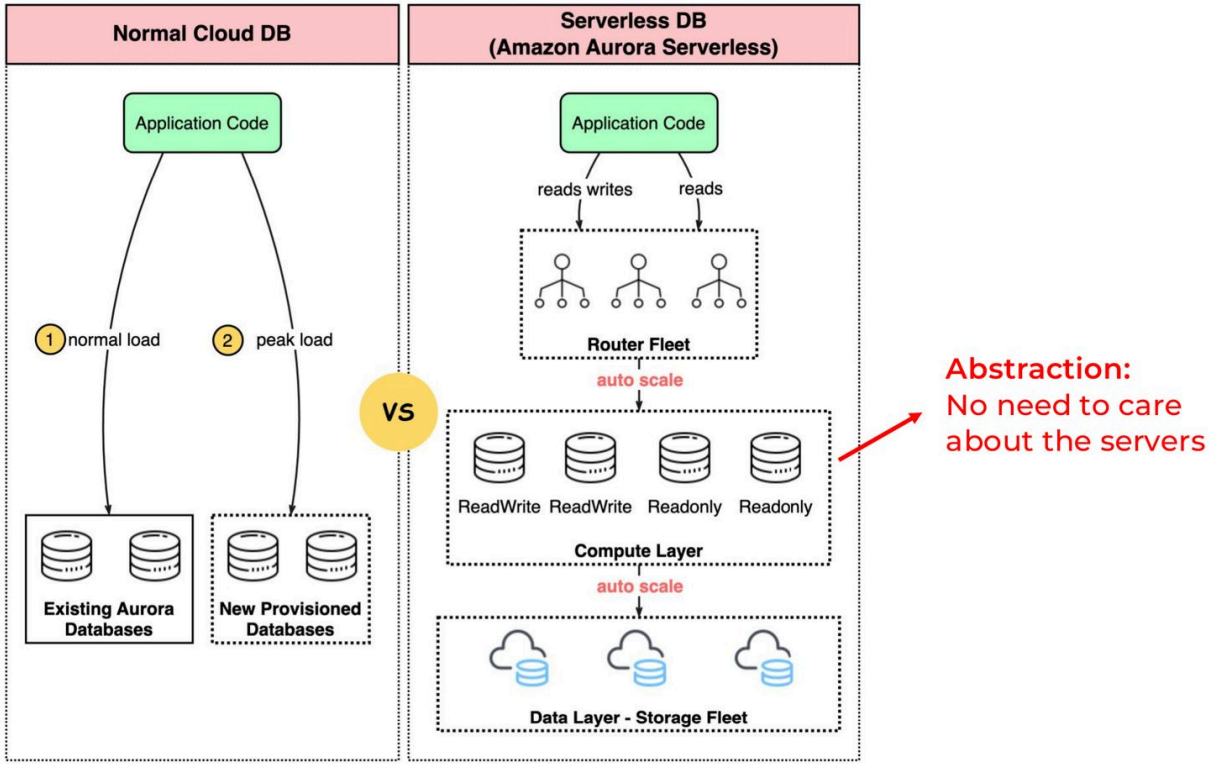
Cloud Service Models



Moving right transfers more infrastructure operations to the provider

Serverless Computing

Serverless transfers server provisioning and operation to a managed service



The service scales servers behind the application interface

Milestones in Managed Analytics

2004
MapReduce paper
batch on clusters

2009
Spark at Berkeley
reusable chains

2010
BigQuery preview
SQL as a service

Today
managed compute
and shared storage

Analytics shifted from operating clusters to submitting SQL to a service

BigQuery as a Managed Service



Ingest

- Batch & Streaming
- Structured & Semi-structured



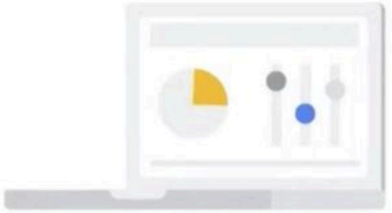
Store

- Up to Petabytes of data
- Datasets & tables
- Federate data



Analyze

- Super-fast SQL queries
- Aggregate
- Transform with DML



Visualize

- Export and connect to other tools more easily

Analysts submit SQL while the service manages distributed execution

Spark and BigQuery

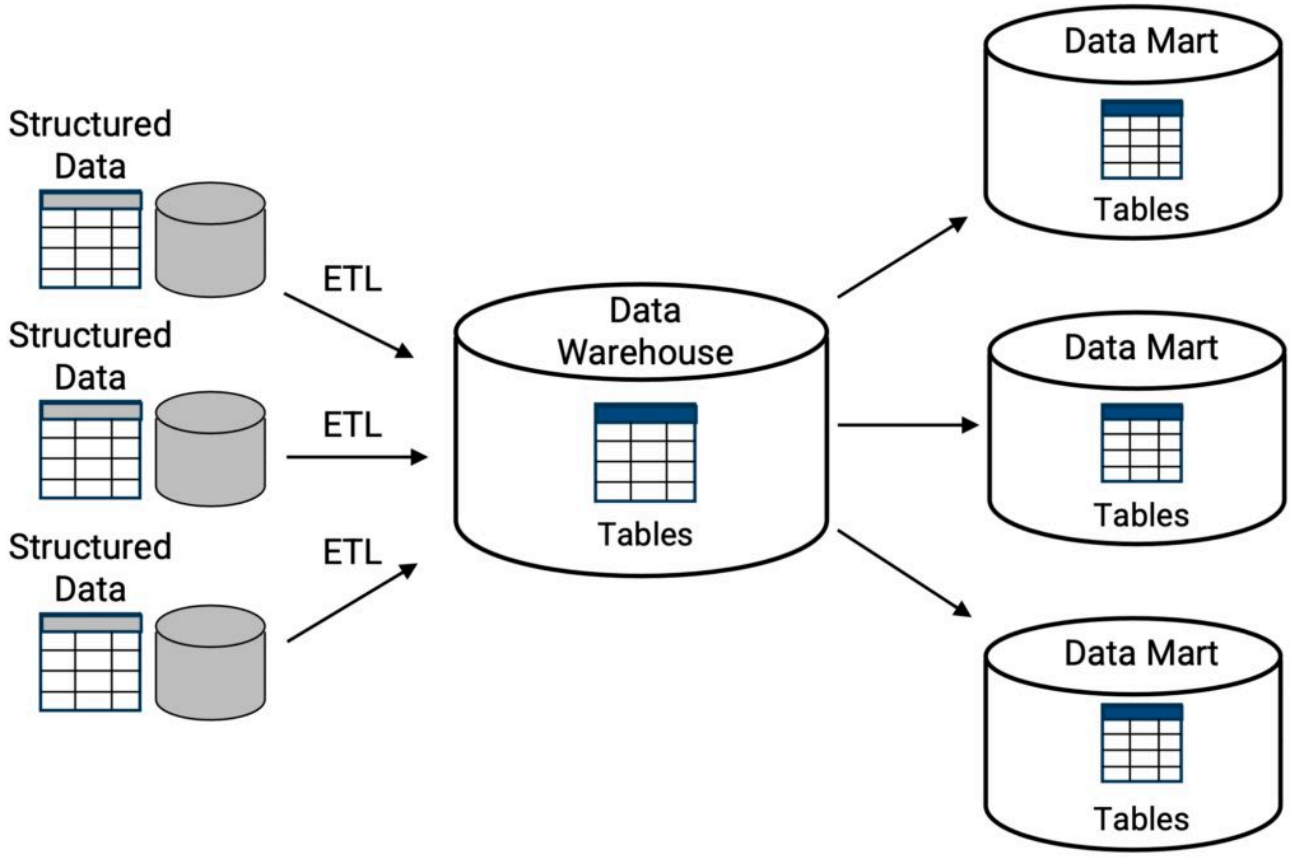
Tool	User expresses	Typical responsibility
Spark	A distributed program	Choose transformations and execution environment
Managed Spark service	A Spark program	Use a provider-managed execution environment
BigQuery	An analytical SQL query	Define tables and the analytical question

An execution engine and a managed analytical service solve different parts of the workload

02 · Analytical Warehouses

01	Cloud and Serverless	Who operates the infrastructure?
02	**Analytical Warehouses**	How are many rows read and summarized?
03	Lakes and Lakehouses	Where do files become tables?
04	Data Orchestration	Which task runs after which, and safely again?

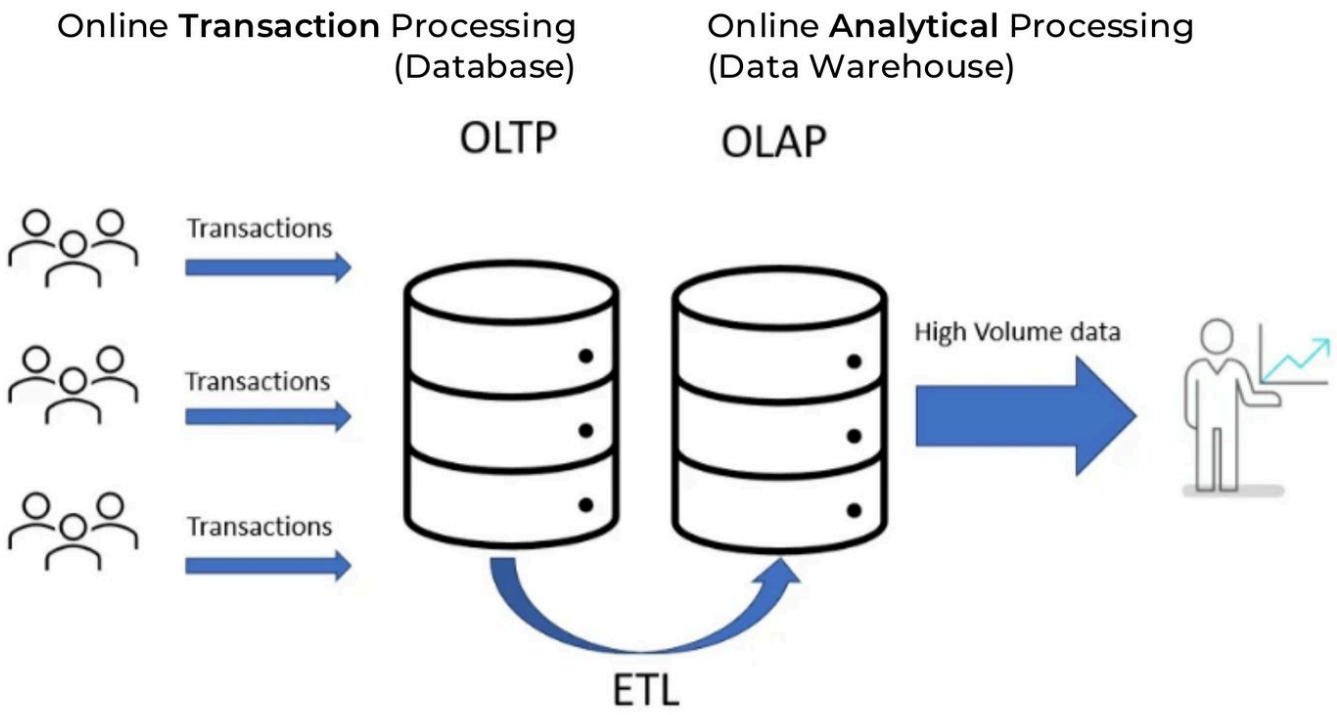
Why an Analytical Warehouse



Shared analytical tables connect multiple sources to consistent departmental reports

OLTP and OLAP

Online Transaction Processing (OLTP) and Online Analytical Processing (OLAP)



Small transactional updates and large analytical scans place different demands on storage

Workloads Shape Physical Storage

Workload	Read / write pattern	Typical physical choice
OLTP	Find and update a few complete records	Row store + key indexes
OLAP	Scan a few columns across many records	Column store + compressed scans
Mixed workload	Fresh writes and large analytical reads	Separate replicas or a hybrid design

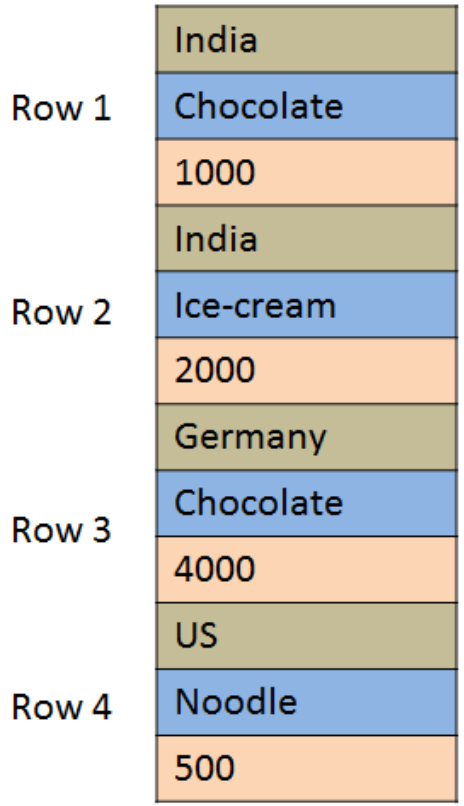
OLTP favors record access; OLAP favors scanning selected attributes

The Same Values on Disk

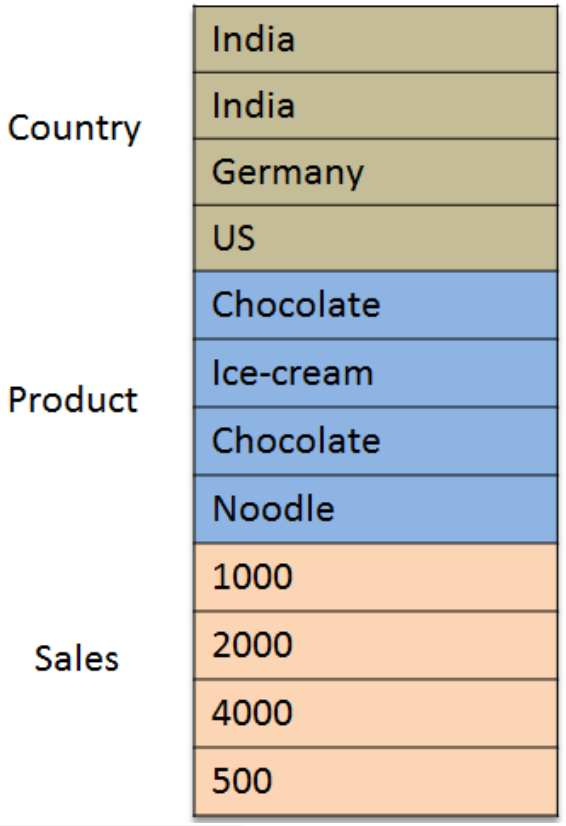
Logical table

Country	Product	Sales
India	Chocolate	1000
India	Ice-cream	2000
Germany	Chocolate	4000
US	Noodle	500

Row store



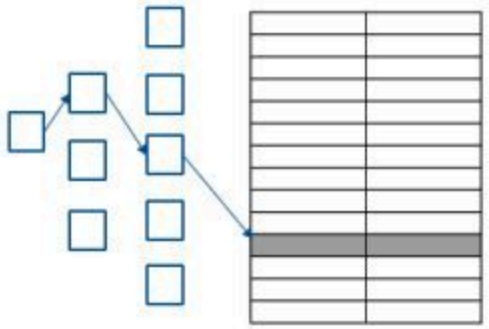
Column store



Twelve values retain their meaning while their physical order changes

Point Reads and Analytical Scans

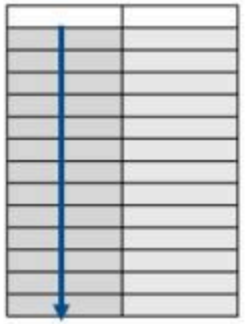
Row-oriented storage & execution



```
SELECT * FROM orders
WHERE orderid = $1
```

At scale: Fast on OLTP, Slow on OLAP

Column-oriented storage & execution



```
SELECT productid, count(*) FROM orders
GROUP BY 1 ORDER BY 2 DESC LIMIT 10
```

At scale: Slow on OLTP, Fast on OLAP

Choose a layout around the values that the workload needs to read

The Tradeoff Is Read and Write Amplification

Operation	Row layout	Column layout
Read one full record	Fetch nearby attributes	Gather values across column segments
Scan 3 of 100 columns	Read unwanted attributes in scanned pages	Read the selected column segments
Frequent record updates	Update record and its indexes	Maintain deltas or rewrite compressed segments

The preferred layout depends on how much data each operation must touch

Columns Enable Compact Encoding

Encoding	One product column	Encoded representation
Dictionary	Chocolate, Ice-cream, Chocolate, Chocolate	0, 1, 0, 0 + dictionary
Run-length	0, 0, 0, 1, 1, 0	(0,3), (1,2), (0,1)

```
SELECT SUM(amount)
FROM sales WHERE product = 'Chocolate';
```

Repeated values and shared types enable encodings that reduce scan input

Prune Columns and Partitions

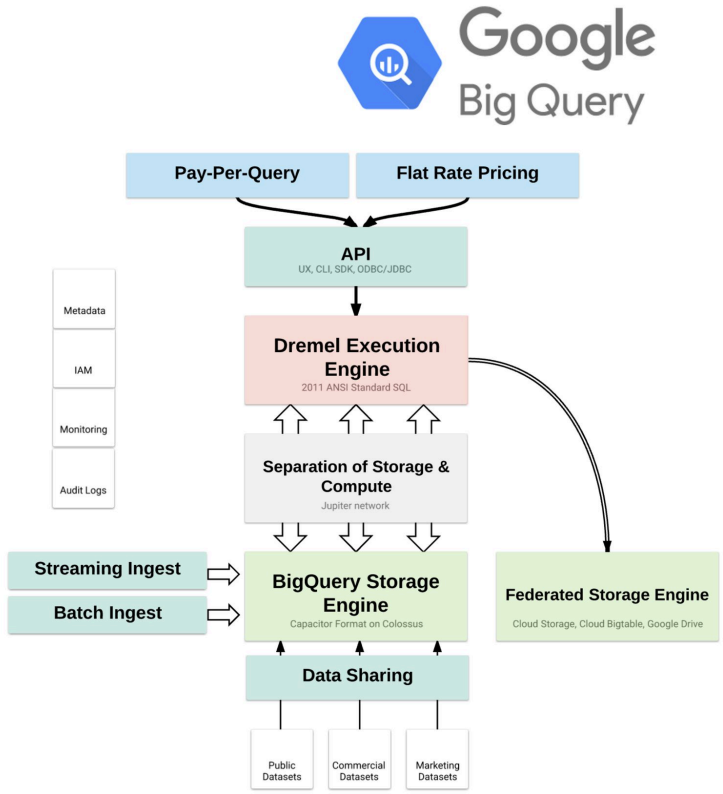
```sql
SELECT product_id, SUM(amount)
FROM sales
WHERE sale_date >= DATE '2025-10-01'
  AND sale_date <  DATE '2025-11-01'
GROUP BY product_id;
```

Projection	Partition filter	Remaining work
product_id, amount, sale_date	October partitions only	Scan, group and combine selected data

Column selection and partition pruning remove different parts of the scan

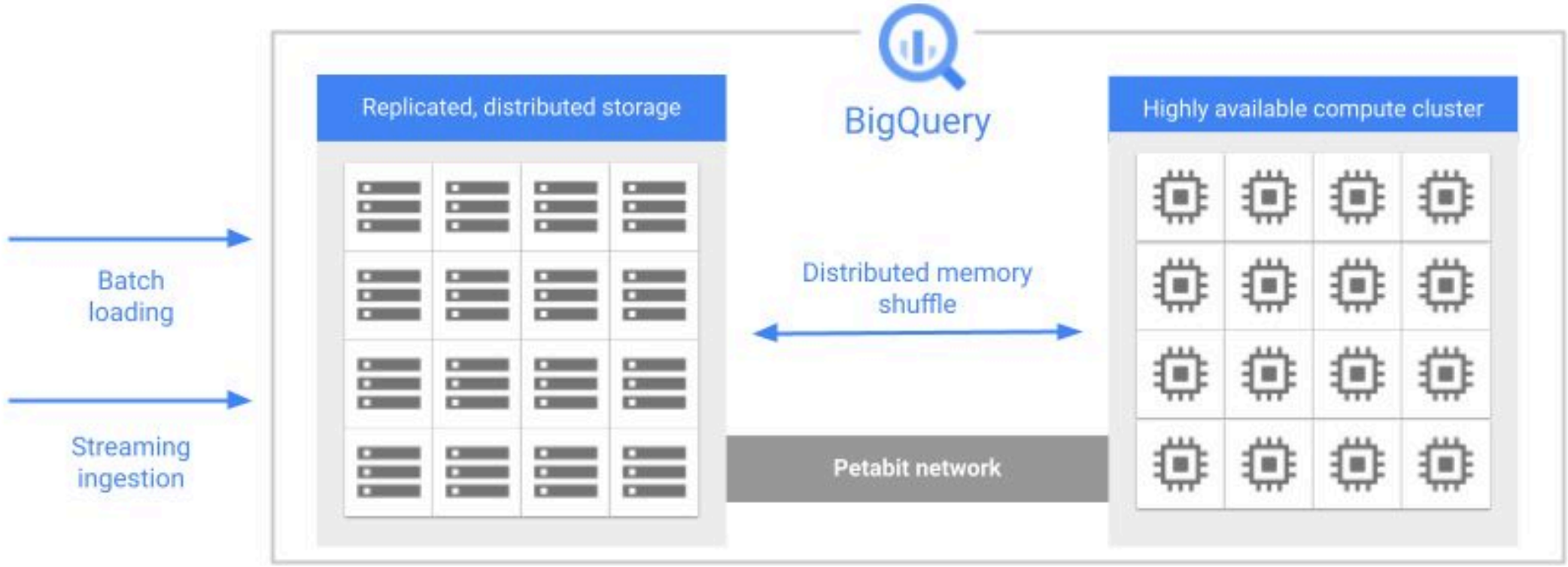
Inside BigQuery

Historical architecture: follow the query and its data



SQL execution, columnar storage and the network form separate service layers

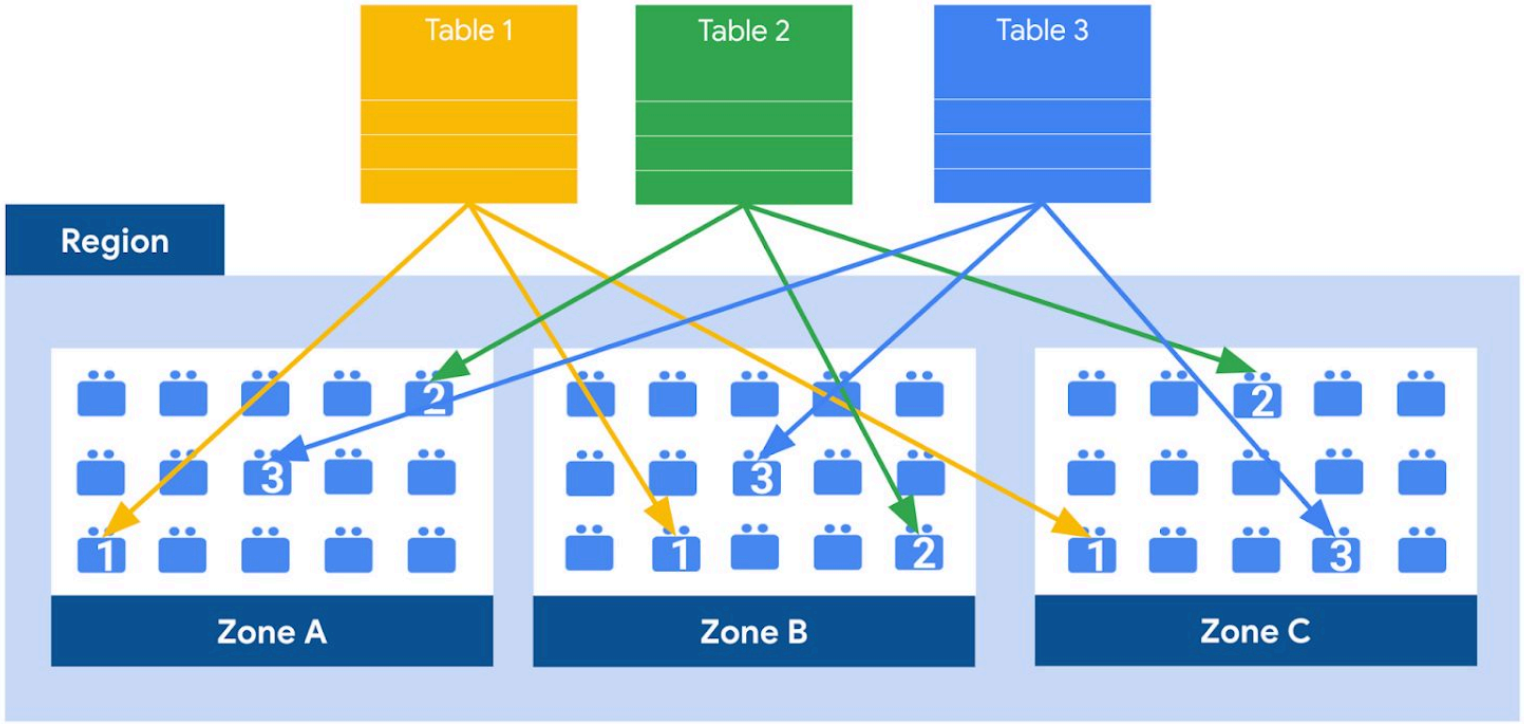
BigQuery Storage and Compute



Persistent table storage and query execution are managed as separate layers

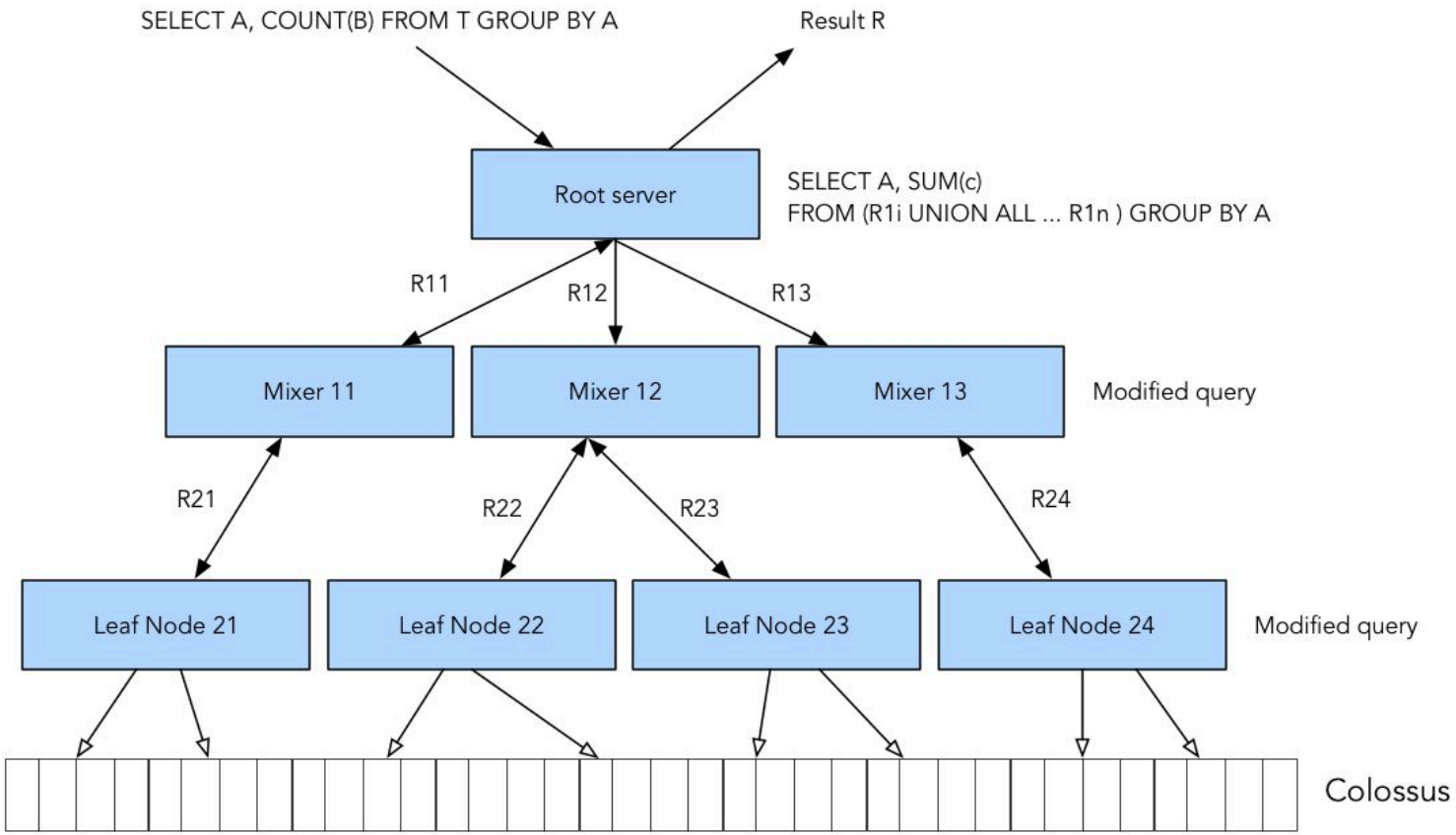
Managed Fault Tolerance

Historical illustration of replicated storage



Replicated storage protects against infrastructure failures rather than incorrect analytical logic

Distributed Query Execution



Workers read portions of the table and combine partial results

Partial Results Must Be Mergeable

```sql
SELECT country, COUNT(*)
FROM sales GROUP BY country;
```

Worker	Partial counts	Root merge
A	India: 2, US: 1	India: 2 + 1 = 3
B	India: 1, US: 3	US: 1 + 3 = 4

Local COUNT results become a global SUM over the same group key

Averages Need Sums and Counts

Worker	Values	Partial state
A	10, 10	sum = 20, count = 2
B	90	sum = 90, count = 1

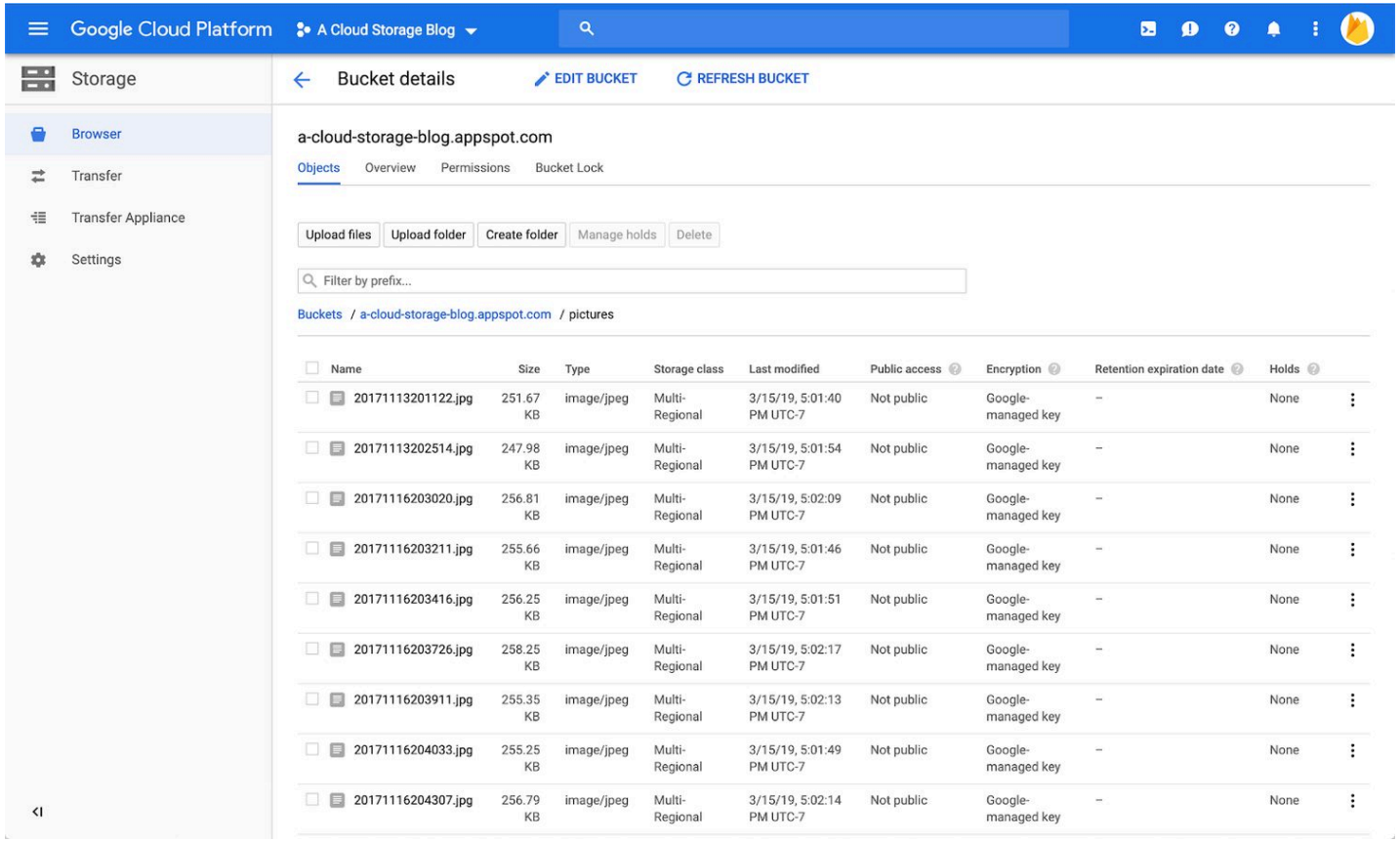
```
AVG = (20 + 90) / (2 + 1) = 36.67
(10 + 90) / 2 = 50    -- wrong: unequal group sizes
```

An aggregate is distributed by preserving the state needed for a correct merge

03 · Lakes and Lakehouses

01	Cloud and Serverless	Who operates the infrastructure?
02	Analytical Warehouses	How are many rows read and summarized?
03	**Lakes and Lakehouses**	Where do files become tables?
04	Data Orchestration	Which task runs after which, and safely again?

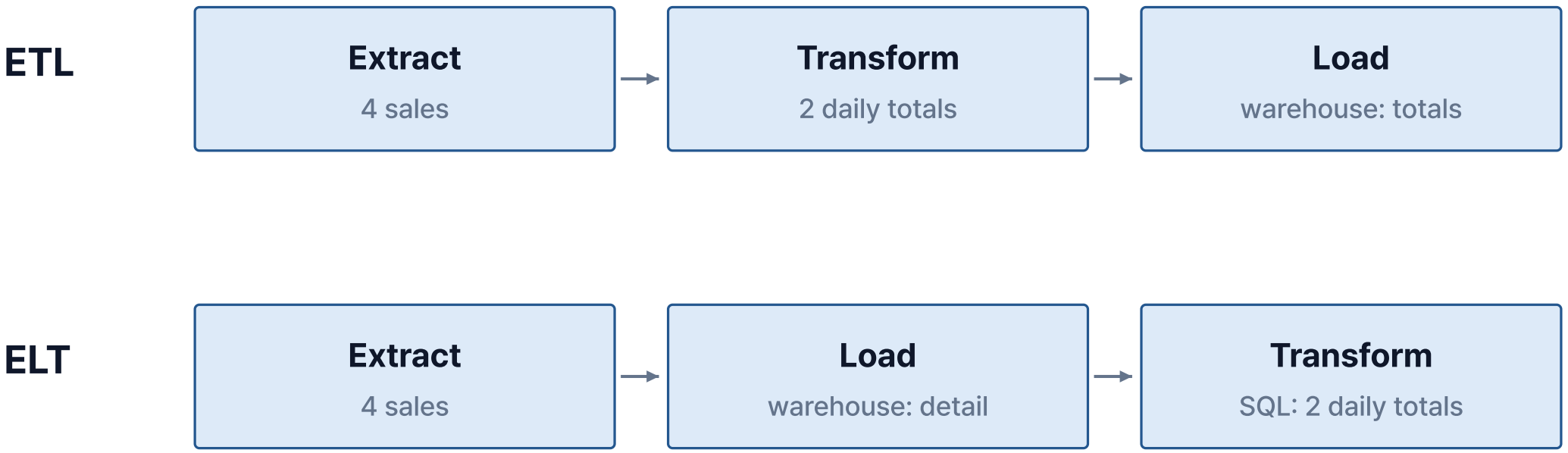
Why Keep a Data Lake



Source files can remain in object storage before analytical tables are defined

ETL and ELT

Extract, Transform, Load (ETL) or Extract, Load, Transform (ELT)



Transform before loading, or load detail first and transform inside the warehouse

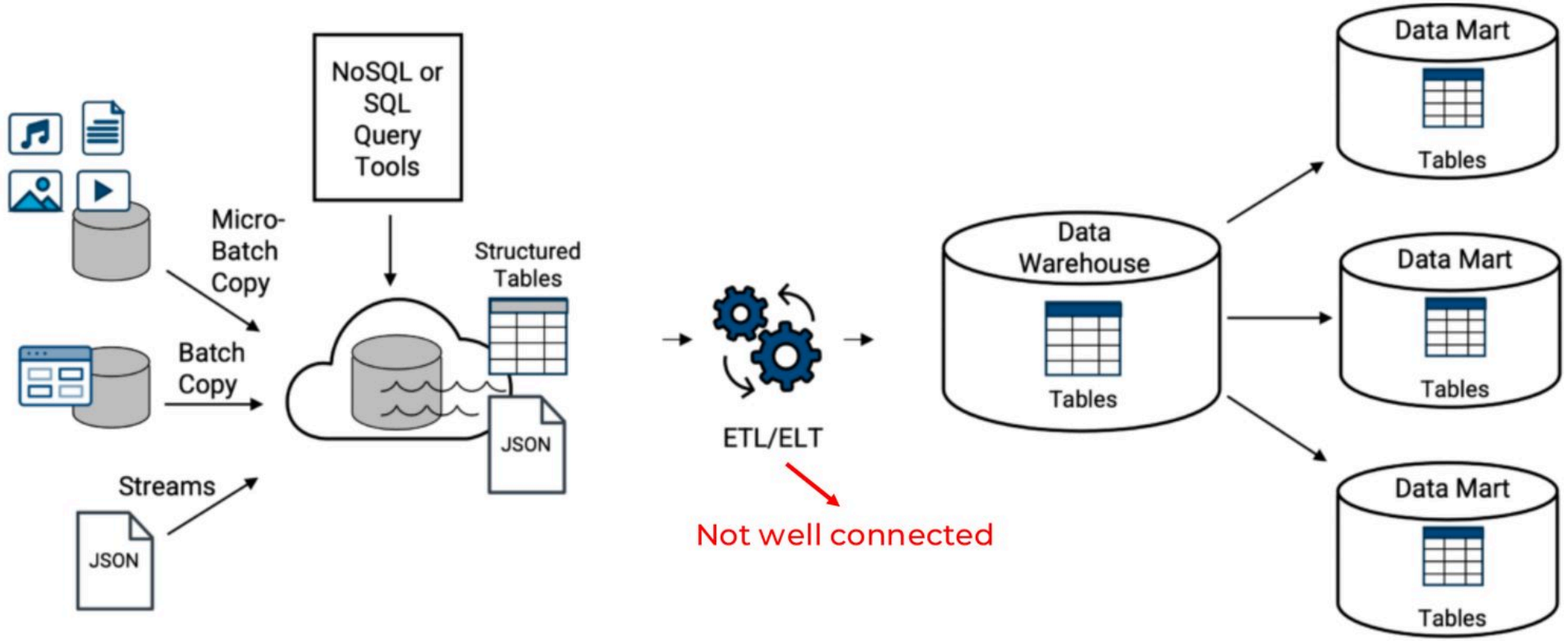
Choosing ETL or ELT

Place transformation where data controls and processing resources require it

Decision	ETL: transform before loading	ELT: load before transforming
Data boundary	Remove identifiers before shared analytics	Keep raw input in a restricted landing area
Compute location	Use upstream compute; reduce target load	Use scalable warehouse SQL compute
Changing analysis	Load a known, validated target shape	Retain detail; rebuild models as needs change

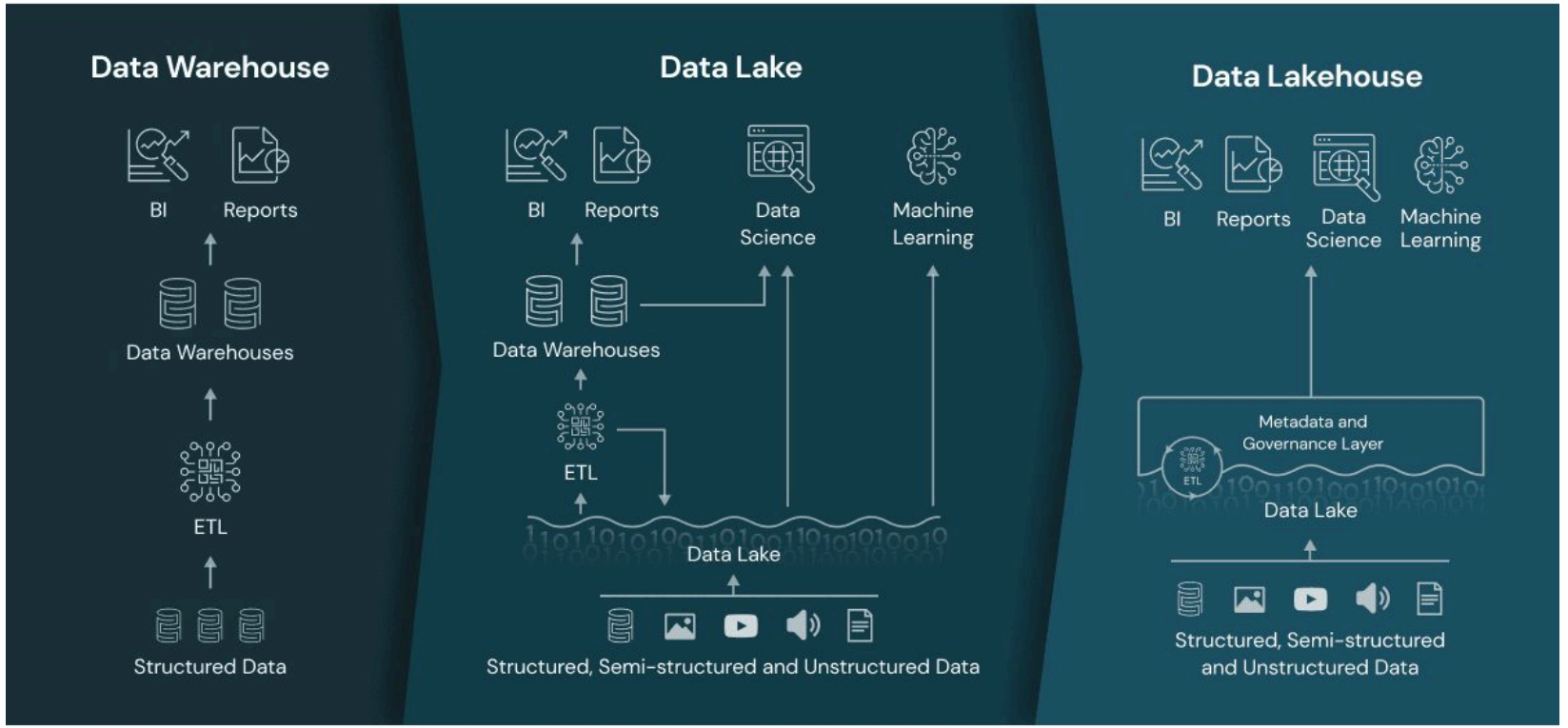
Choose by data boundaries, available compute and the need to reprocess

From Lake to Warehouse



Separate raw and analytical stores introduce copying, transformations and refresh dependencies

Why a Lakehouse



Lakehouse adds table management over lake files for shared analytical use

What Makes a Lakehouse Work

Layer	Mechanism	Reason
Table management	Transactions and versioned metadata	Readers see a consistent table
Query execution	Columnar files, pruning and statistics	SQL scans avoid unrelated data
Shared access	SQL and DS/ML use the same snapshot	Fewer independent data copies

Table transactions, query execution and shared data access turn files into managed tables

Commit a Table Snapshot

Legacy Non-Transactional Hive Table

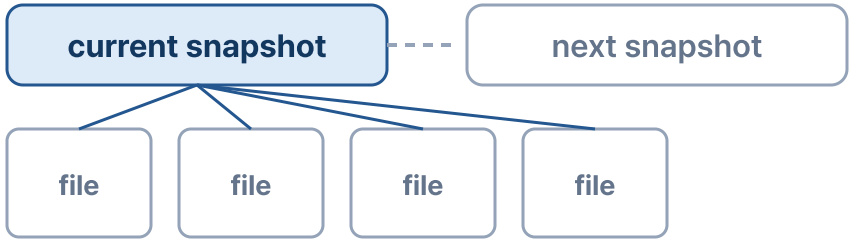


directory-based file discovery

no table-wide snapshot commit

readers can see a write in progress

Iceberg Table = Metadata



the snapshot lists its files

an atomic metadata commit swaps the pointer

readers see one complete snapshot

An atomic metadata commit publishes a complete file set to readers

Tools Around a Lakehouse

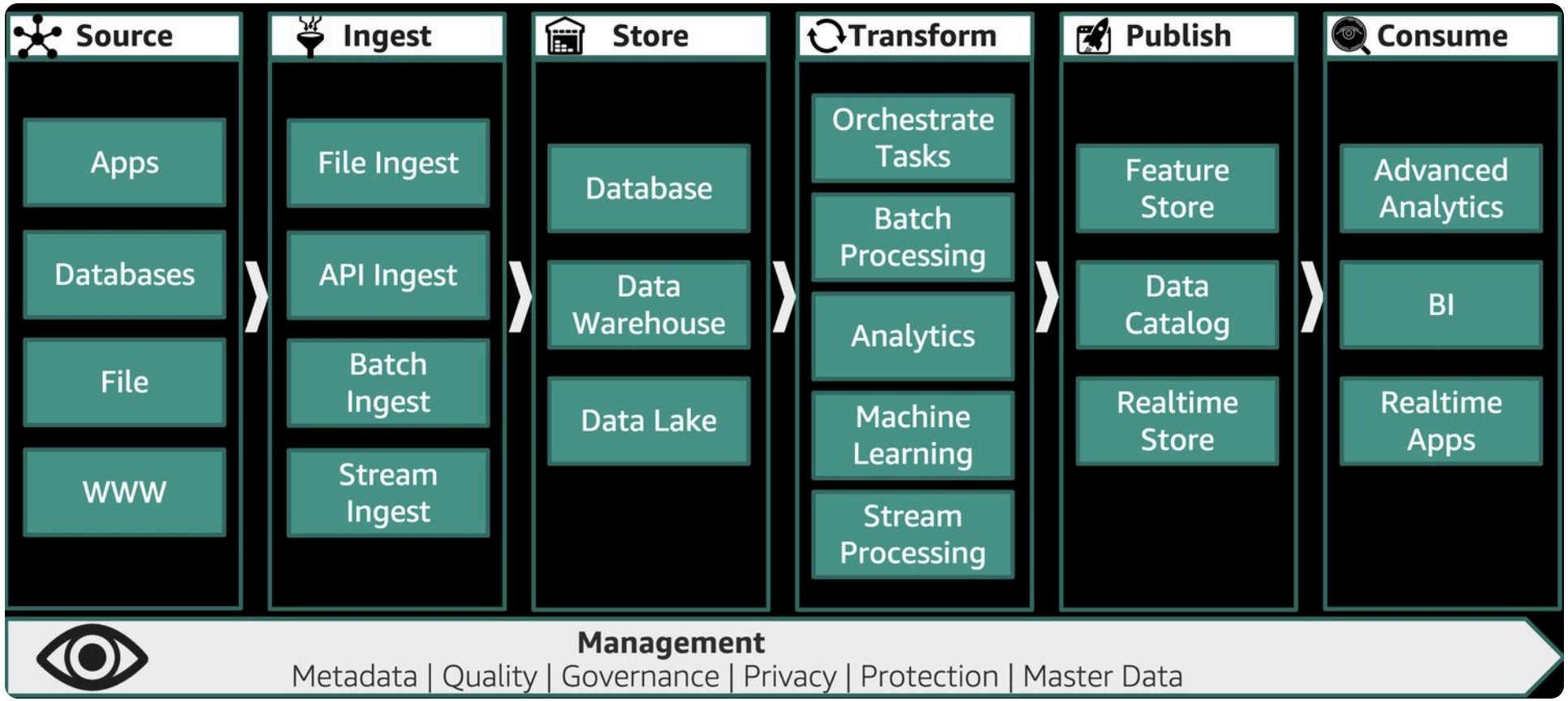
Compute and platform	**Spark, Databricks**	run transformations and queries
Table format	**Iceberg, Delta Lake**	manage table metadata and changes
Columnar file format	**Parquet**	encode values in files
Object storage	**S3, Cloud Storage**	store the data files

Storage, file format, table format and compute belong to different layers

04 · Data Orchestration

01	Cloud and Serverless	Who operates the infrastructure?
02	Analytical Warehouses	How are many rows read and summarized?
03	Lakes and Lakehouses	Where do files become tables?
04	**Data Orchestration**	Which task runs after which, and safely again?

Why Data Orchestration



A successful query can still run before its required data is ready

Airflow at Airbnb

2014
started at Airbnb by M. Beauchemin

2015
announced as open source

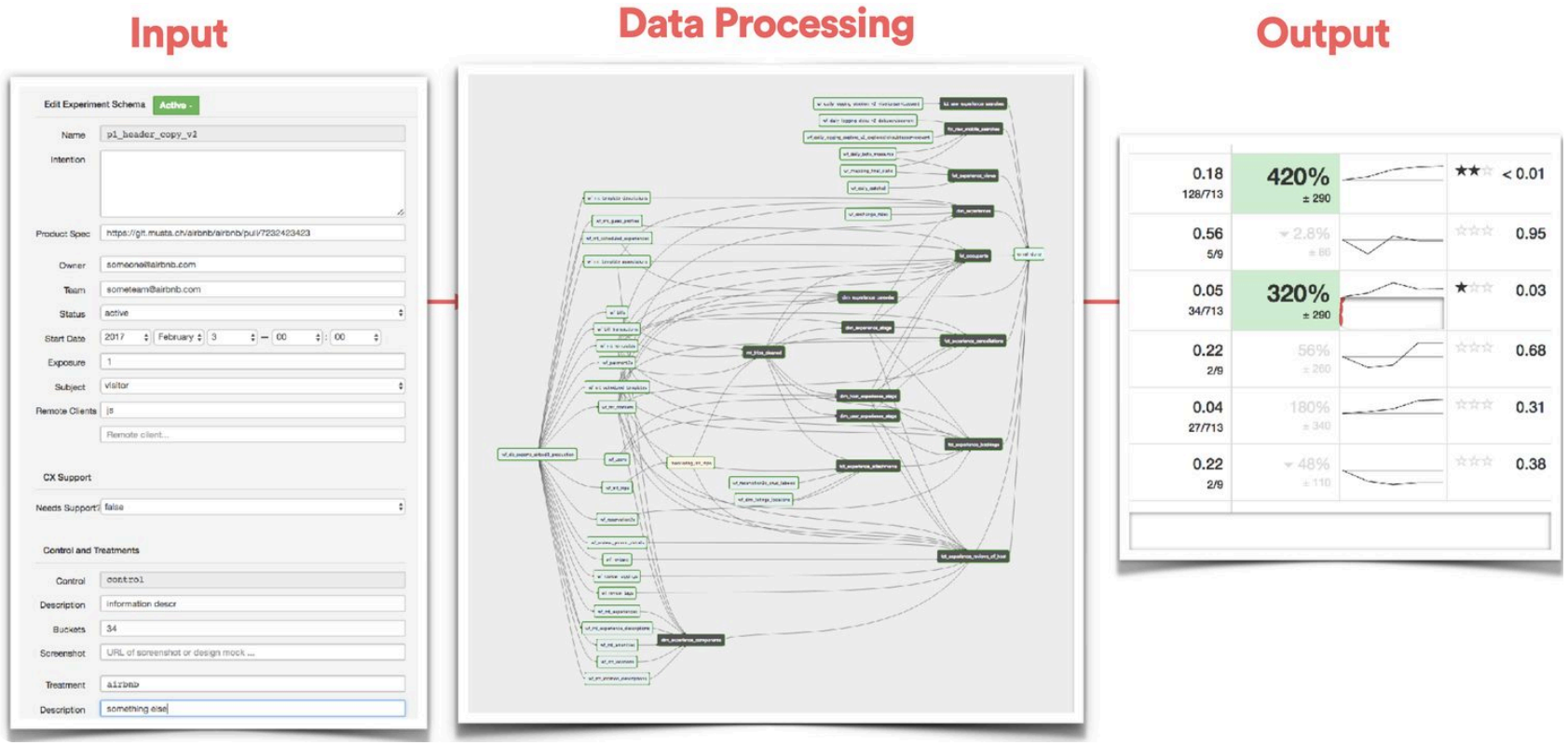
2016
Apache incubation

2019
Apache top-level project

Airflow grew from a company workflow tool into an Apache project

Case study: Airbnb Experiment Analytics

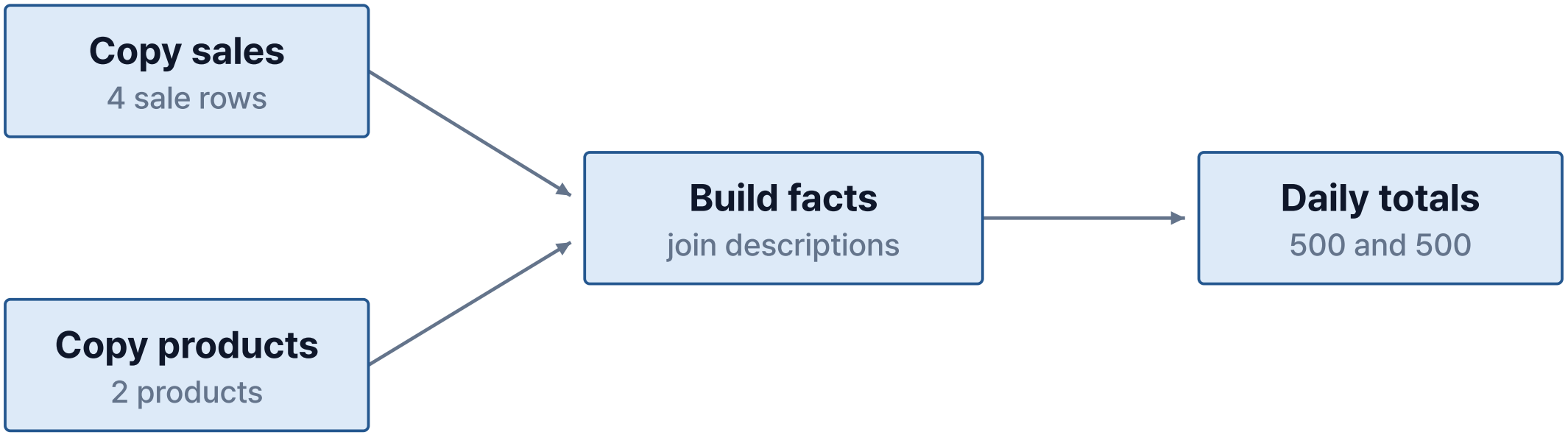
Historical Airbnb workflow: configuration, computation and experiment metrics



Experiment configuration drives a dependency graph that produces analytical results

A Sales Workflow DAG

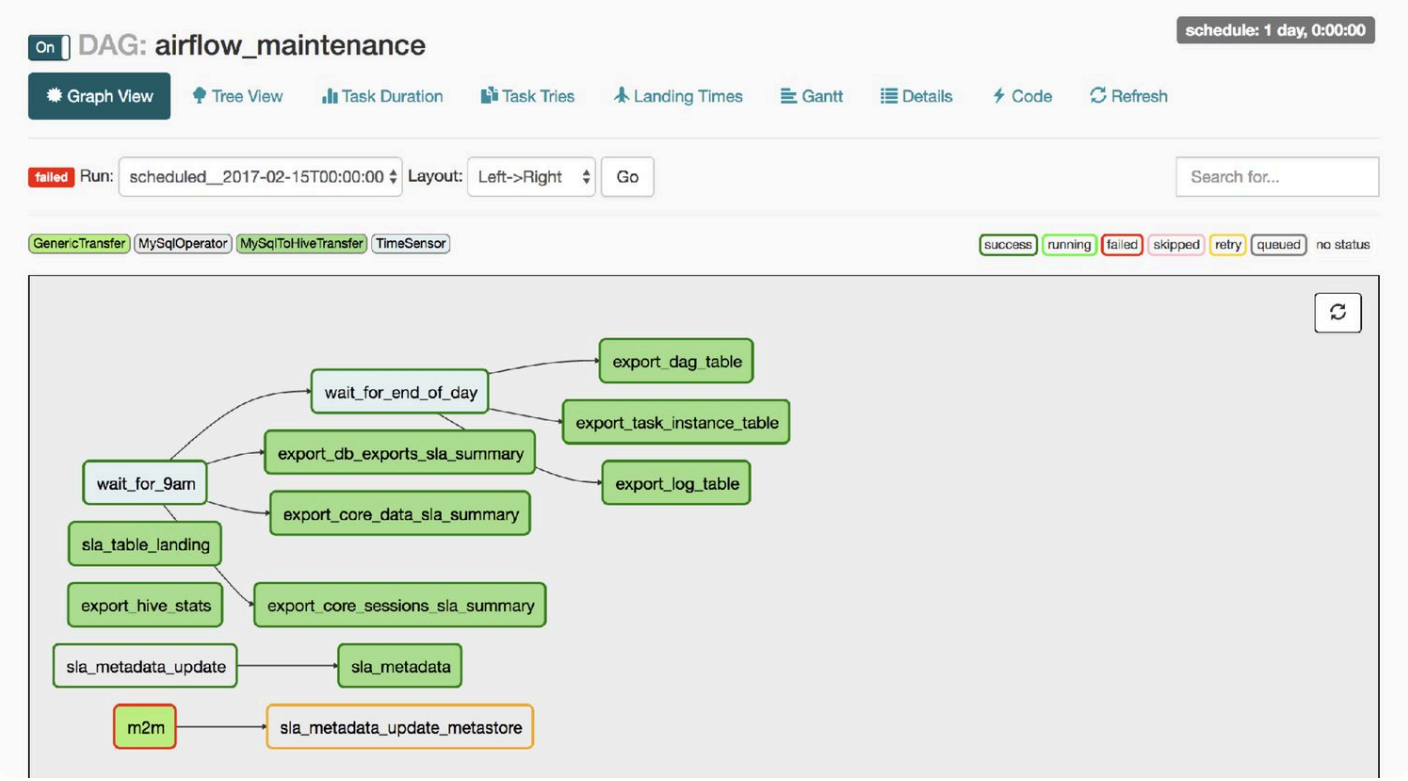
A Directed Acyclic Graph (DAG) defines tasks and their dependencies



Sales and products can load independently; the join waits for both inputs

Read a Real Dependency Graph

Historical Airflow maintenance DAG



Independent branches can progress while descendants of a failed task remain blocked

Tasks, Operators and Sensors

Concept	Meaning	Sales workflow example
Task	One defined unit of work	Load the sales file
Operator	A template used to create a task	Submit a SQL query
Sensor	A waiting task pattern	Wait for the input file

A workflow combines concrete tasks built from reusable execution and waiting patterns

Define the Run Policy

Airflow 2.10 example: policy inside the DAG definition

```
from airflow import DAG
import pendulum
from datetime import timedelta

with DAG("daily_sales",
    start_date=pendulum.datetime(2025, 10, 2, tz="UTC"),
    schedule="@daily", catchup=False,
    default_args={"retries": 2,
                  "retry_delay": timedelta(minutes=5)}) as dag:
    ...
```

A schedule creates run instances; retries apply to tasks within each run

Wait for Complete Input, Then Run SQL

Airflow 2.10 + common-sql 1.26; connections and SQL file configured separately

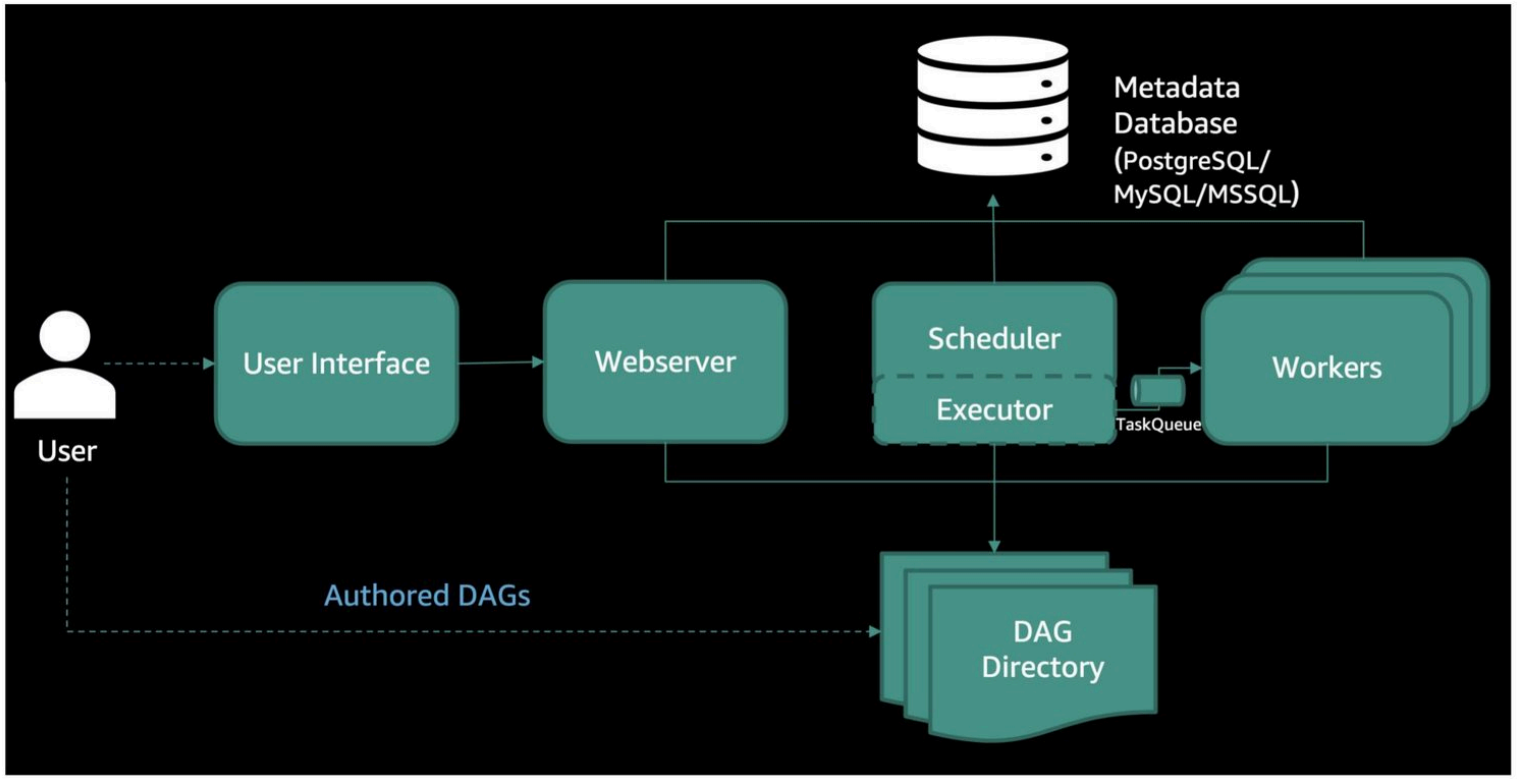
```
from airflow.sensors.filesystem import FileSensor
from airflow.providers.common.sql.operators.sql import SQLExecuteQueryOperator

# Inside the DAG context from the previous slide
wait = FileSensor(task_id="wait_for_sales",
    filepath="sales/{{ data_interval_start | ds }}/_SUCCESS",
    fs_conn_id="sales_files", mode="reschedule", timeout=3600)
load = SQLExecuteQueryOperator(task_id="load_daily",
    conn_id="warehouse", sql="load_daily.sql")
wait >> load
```

A sensor checks readiness; the downstream operator submits the transformation

Airflow Control and Workers

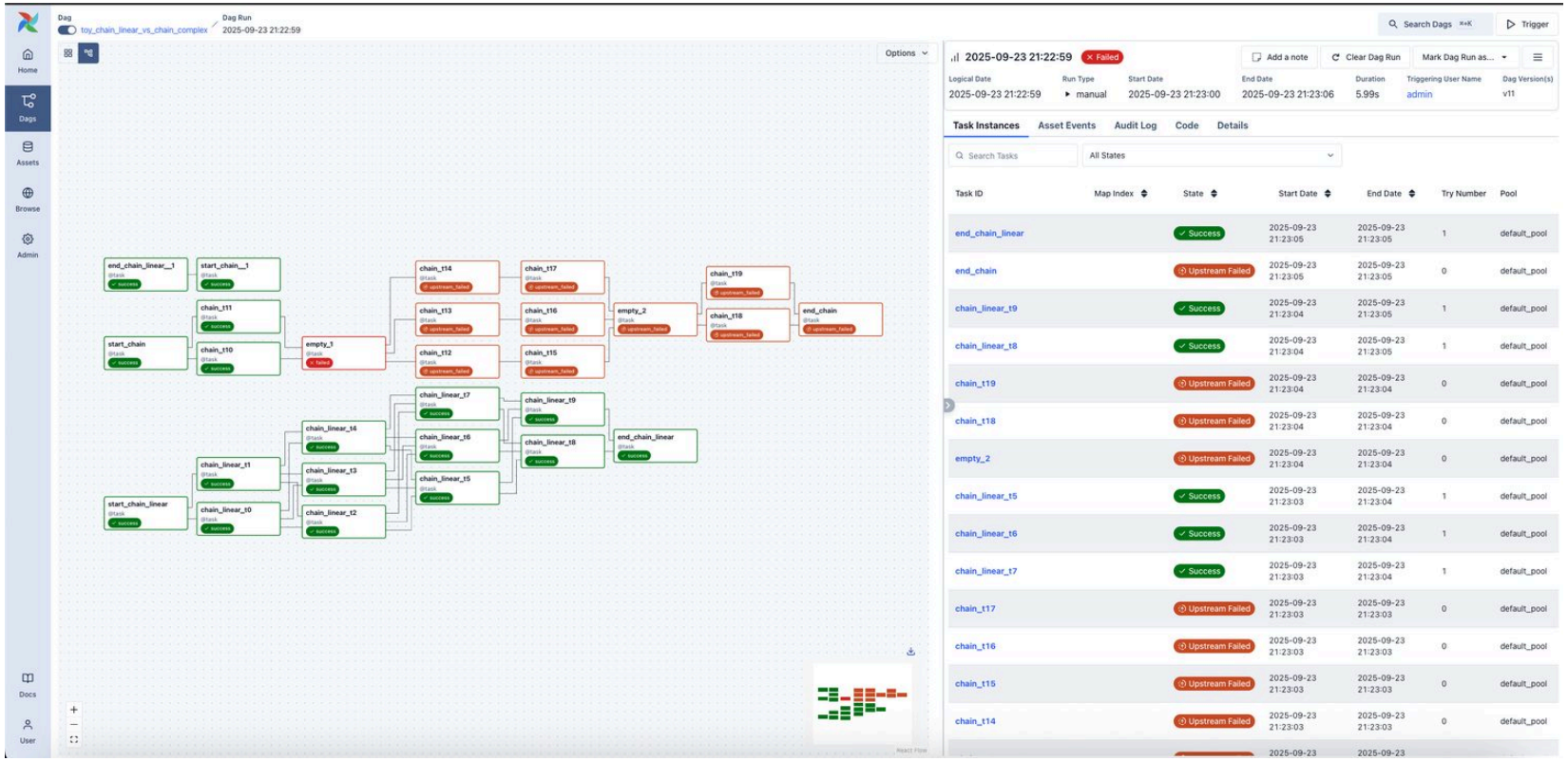
Historical architecture: focus on the responsibilities, rather than deployment details



Scheduling, task execution and recorded state are distinct responsibilities

An Airflow Run

Apache Airflow interface



A real run shows succeeded, failed and waiting tasks

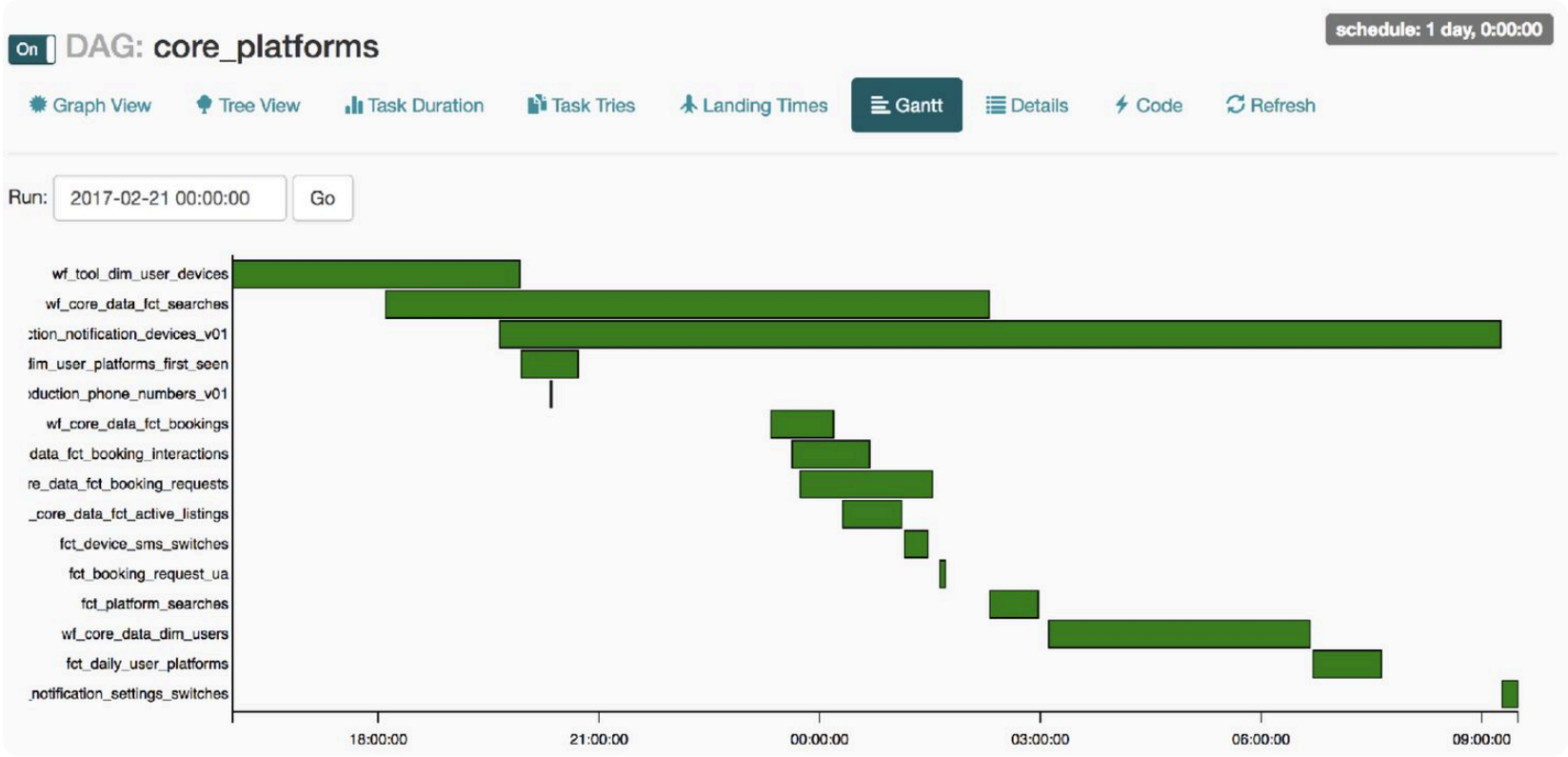
Caveat: An Upstream Failure



A failed product load prevents the dependent join from using incomplete inputs

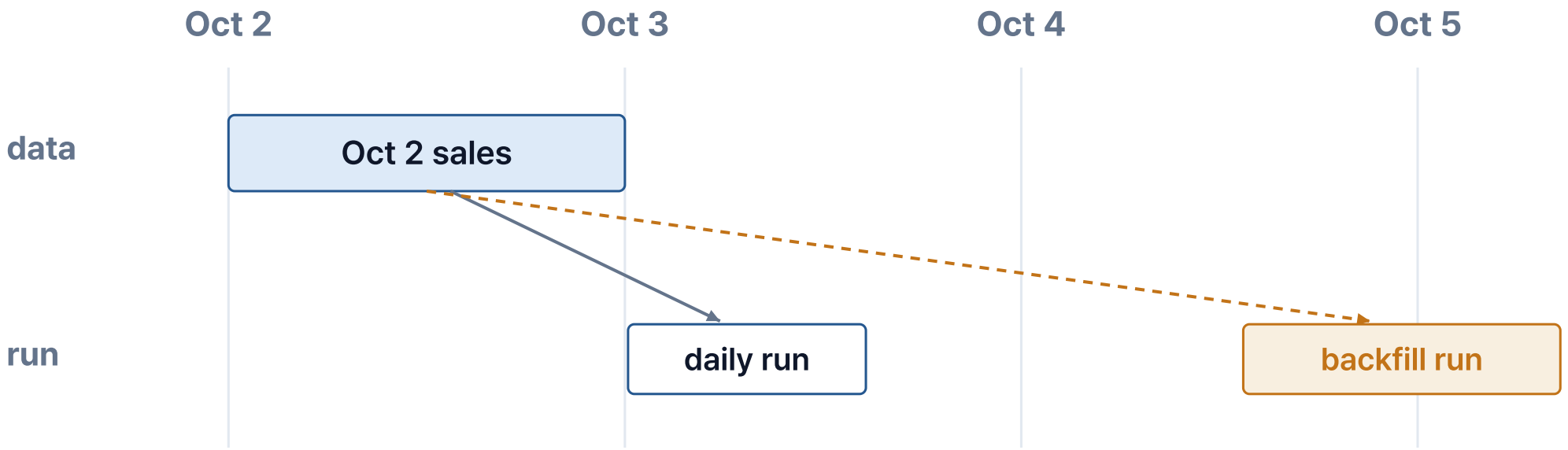
Read Execution Time in Gantt

Historical Airflow Gantt: actual overlap and long-running tasks



Use task durations together with dependencies to locate the path delaying completion

Data Interval and Run Time



The date being recomputed can differ from the time when the task runs

Catchup, Backfill and Retry

Mechanism	Unit of work	What stays fixed
Retry / clear task	A task instance in an existing run	Its data interval
Catchup	Missing scheduled intervals	The DAG timetable
Backfill	Selected historical intervals	The transformation applied to each interval

Re-execution changes when work runs, rather than which data interval it should read

Recap: Inputs for a Retried Load

Source fact_sales	Date	Amount
sale 1, P1	2025-10-02	300
sale 2, P2	2025-10-02	200
sale 3, P1	2025-10-03	400
sale 4, P2	2025-10-03	100

Target daily_sales already contains October 2 = 500; load October 3 without duplication

Caveat: Retrying an Append

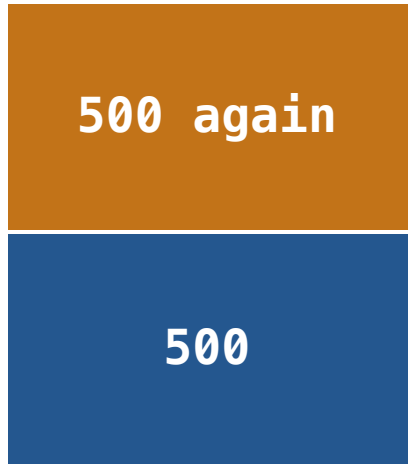
Append once

500



Append twice

1000



Retrying the same append can count the same sales twice

Demo: Repeatable Date Load

Local SQLite exercise: safe task behavior under retries

1. Run the supplied date replacement twice in one script
2. Compare it with appending the same date twice

```
BEGIN;
DELETE FROM daily_sales WHERE
sale_date='2025-10-03';
INSERT INTO daily_sales
SELECT sale_date,SUM(amount) FROM fact_sales
WHERE sale_date='2025-10-03' GROUP BY
sale_date;
COMMIT;
SELECT * FROM daily_sales ORDER BY sale_date;
```

QUERY RESULT

sale_date	total
2025-10-02	500
2025-10-03	500

Answer: Repeatable Date Load

```
BEGIN;
DELETE FROM daily_sales WHERE sale_date='2025-10-
03';
INSERT INTO daily_sales
SELECT sale_date,SUM(amount) FROM fact_sales
WHERE sale_date='2025-10-03' GROUP BY sale_date;
COMMIT;
-- run the same block again
SELECT * FROM daily_sales ORDER BY sale_date;
```

sale_date	total
2025-10-02	500
2025-10-03	500

Idempotent load: rerunning the same date range leaves the same result

Caveat: A Green DAG Can Hide a Failure

```
transform [failed]  ->  cleanup [success]
                          trigger_rule="all_done"
```

Check	What it actually establishes
Run status	Terminal states of leaf tasks
Task success	The task returned successfully
Data checks before publish	Required properties of the produced data

Run state, task execution and analytical correctness are separate guarantees

Takeaway: Tools Have Different Responsibilities

1. **Row and column layouts** follow transactional and analytical access patterns
2. **Analytical engines** prune scans, merge partial results and read consistent tables
3. **Orchestration** ties dependencies and retries to a defined data interval